

Week 10 – Control of Manipulators

ELEC0129 Introduction to Robotics

Dr. Chow Yin Lai

Email: uceecyl@ucl.ac.uk

Schedule

Legend:

A: MPEB 6.02 Teaching Lab

B: Roberts 7.04 PC Lab

Week	Recorded, uploaded by Friday of previous week	Workshop, Mondays 11am-1pm and/or Wednesdays 9am-11am	Notes
1	(Scenario Week)		
2	Lec: Intro; Spatial description	A: Workshop: Offline programming	Half class Mon, half class Wed
3	Lec & Tut: Spatial description	A: Workshop: Build robot	
4	Lec & Tut: Forward kinematics	A: Workshop: Forward kinematics	
5	Lec & Tut: Inverse kinematics	B: Workshop: Offline programming	Half class Mon, half class Wed
RW	(Reading Week)		
6	(Scenario Week)		
7	Lec & Tut: Jacobians	A: Workshop: Inverse kinematics	
8	Lec: Trajectory Planning	A: Workshop: Trajectory planning	
9	Lec & Tut: Dynamics	A: Workshop: Trajectory planning	
10	Lec & Tut: Control	A: Workshop: Pick-and-place demo	

Content

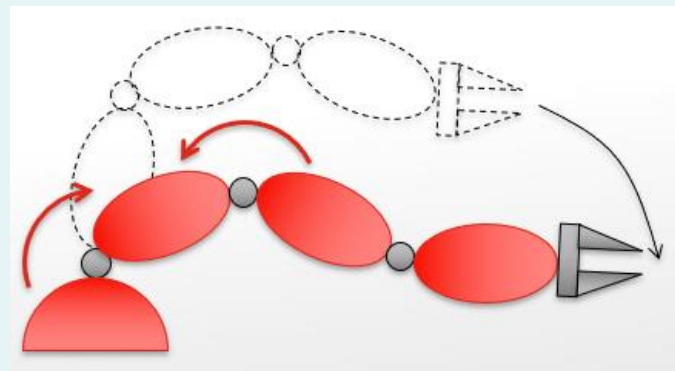
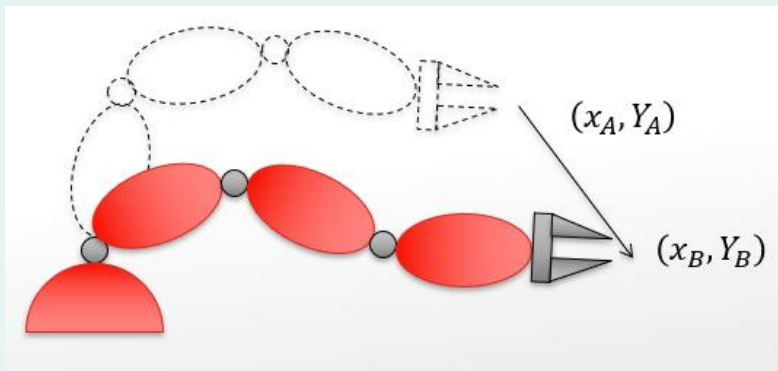
- Introduction
- Second Order Linear Systems
- Control of Second Order Linear Systems
- Control Law Partitioning
- Trajectory Following
- Modeling and Control of a Single Joint
- Modeling and Control of Complete Manipulator
- Matlab Simulation
- Tutorial Questions

Content

- Introduction
- Second Order Linear Systems
- Control of Second Order Linear Systems
- Control Law Partitioning
- Trajectory Following
- Modeling and Control of a Single Joint
- Modeling and Control of Complete Manipulator
- Matlab Simulation
- Tutorial Questions

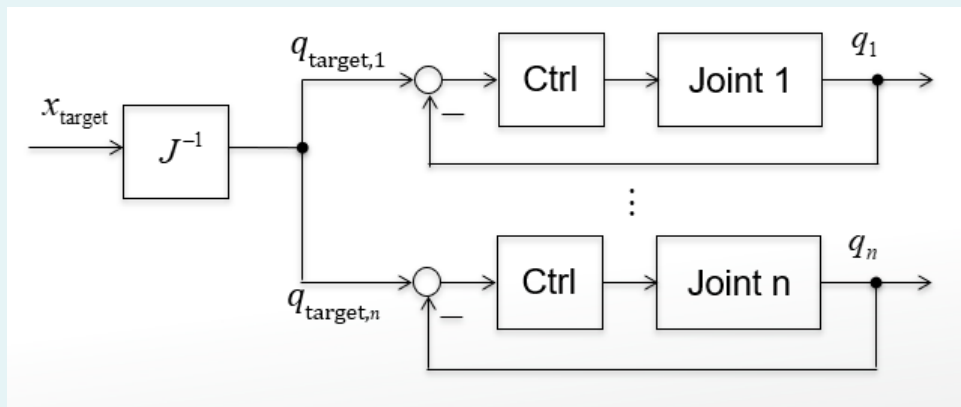
Introduction (1)

- Last week, we discussed about the **trajectory** which the robot is required to follow.
- Today, we will study how we can **control the robot (or joints)** so that they follow the desired trajectory.



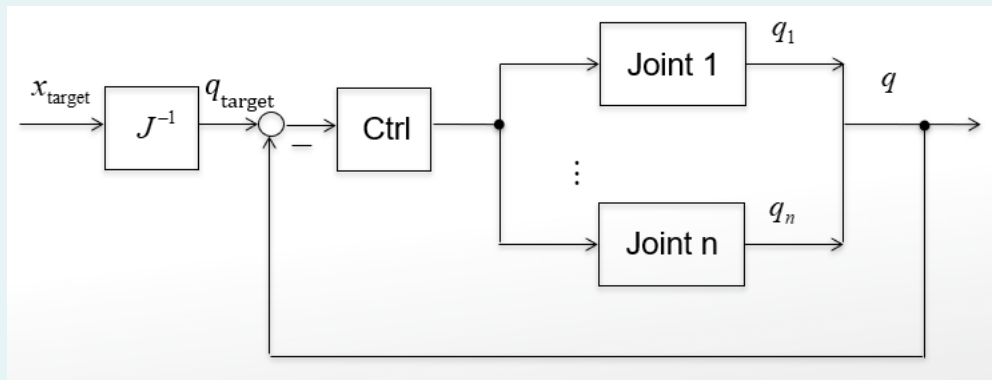
Introduction (2)

- These can be achieved via two different techniques.
- The first technique approximates the nonlinear, coupled manipulator as a few **linear and decoupled joints/links**, and we **control each joints individually**.



Introduction (3)

- The second technique **views the manipulator as a whole**, takes the joint readings as a vector, and generates the control signal also as a vector.
- The **performance is better** because the nonlinearities and coupling are taken into consideration.

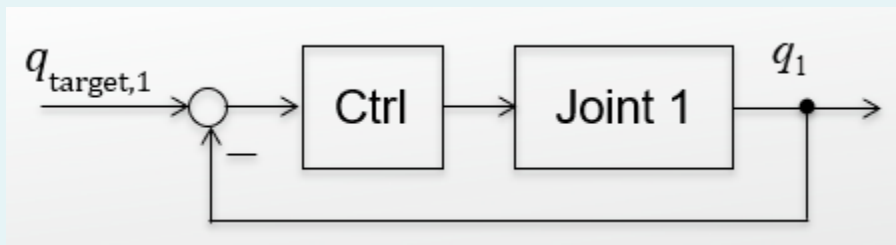


Content

- Introduction
- **Second Order Linear Systems**
- Control of Second Order Linear Systems
- Control Law Partitioning
- Trajectory Following
- Modeling and Control of a Single Joint
- Modeling and Control of Complete Manipulator
- Matlab Simulation
- Tutorial Questions

Natural Systems

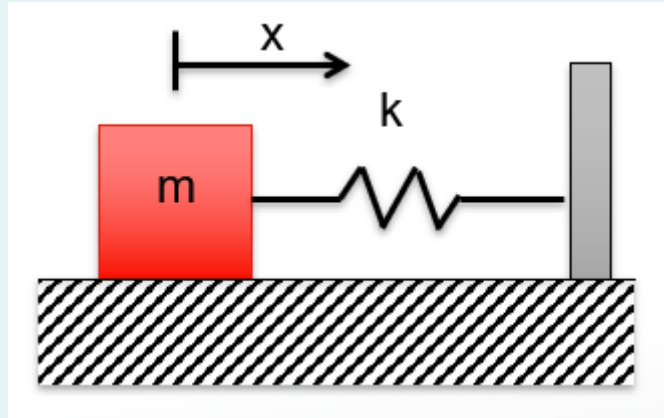
- We will first explore the **individual joint control** technique due to its simplicity.



- But before that, let's discuss about **natural systems** first.
 - The understanding of this will help us with the control design later.

Mass-Spring System (1)

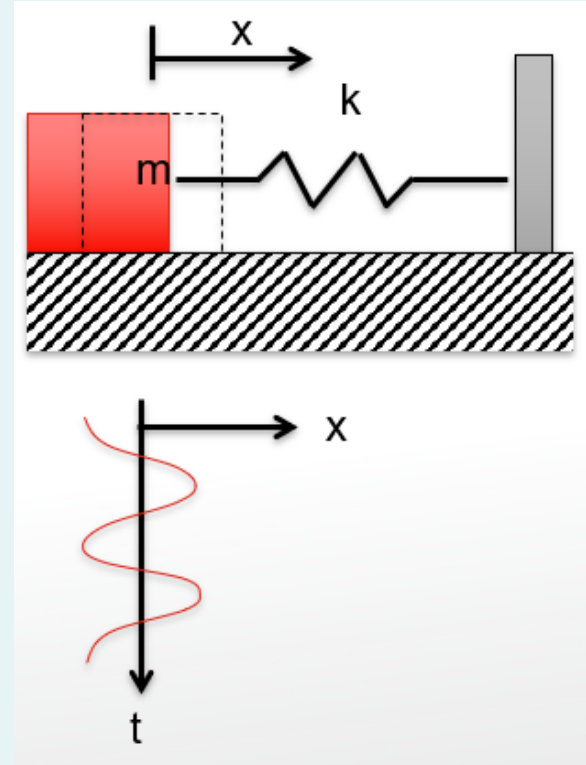
- Imagine you have a mass-spring-system on a frictionless surface:



- Let the equilibrium position be $x = 0$.

Mass-Spring System (2)

- If you perturb the mass from its equilibrium position, and then release it, the mass will **swing back and forth** continuously.



Mass-Spring System (3)

- This can be predicted using mathematics!
- The differential equation (Newton's Law) for the mass-spring system on frictionless surface is:

$$m\ddot{x} = \sum \text{forces} = -kx \Rightarrow m\ddot{x} + kx = 0$$

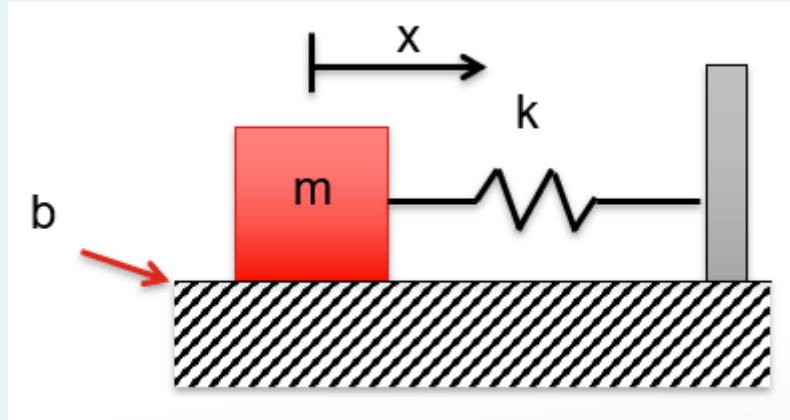
- Solving the differential equation gives:

$$x = C_1 \cos\left(\sqrt{\frac{k}{m}}t\right) + C_2 \sin\left(\sqrt{\frac{k}{m}}t\right)$$

- Where $\omega_n = \sqrt{\frac{k}{m}}$ is the natural frequency of the system.

Mass-Spring-Damper System (1)

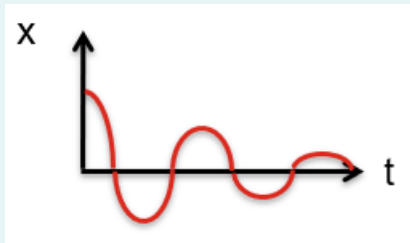
- Now, imagine the surface is not frictionless anymore.
- Instead it has a **viscous friction b** , which acts like a damper.



- Let the equilibrium position be $x = 0$.

Mass-Spring-Damper System (2)

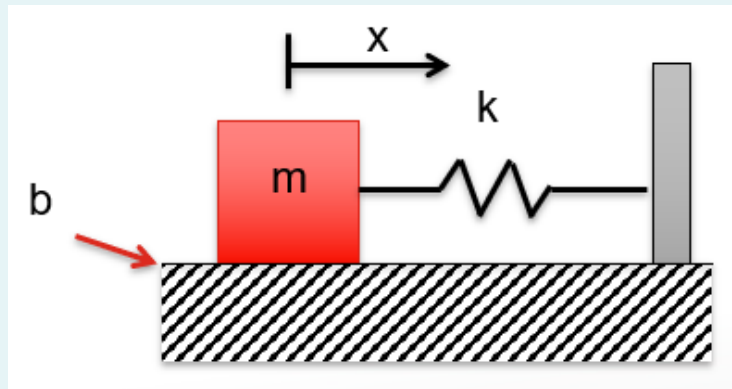
- If you perturb the mass from its equilibrium position, and then release it, there are **three possible responses** depending on **m, b and k** .
 - Overdamped response,
 - Underdamped response.
 - Critically damped response.



Mass-Spring-Damper System (3)

- What does mathematics say about these responses?
- The **differential equation** (Newton's Law) for the mass-spring system on surface with viscous friction is:

$$m\ddot{x} = \sum \text{forces} = -b\dot{x} - kx \Rightarrow m\ddot{x} + b\dot{x} + kx = 0$$



Mass-Spring-Damper System (4)

- The characteristic equation of the 2nd order differential equation is:

$$m\lambda^2 + b\lambda + k = 0$$

- and the roots are:

$$\lambda_1 = \frac{-b + \sqrt{b^2 - 4mk}}{2m}, \lambda_2 = \frac{-b - \sqrt{b^2 - 4mk}}{2m}$$

Overdamped Response

- If $b^2 > 4mk$, we will have real and unequal roots.

$$\lambda_1 = \frac{-b + \sqrt{b^2 - 4mk}}{2m}, \lambda_2 = \frac{-b - \sqrt{b^2 - 4mk}}{2m}$$

- The general solution of the 2nd order ODE is:

$$x(t) = c_1 e^{\lambda_1 t} + c_2 e^{\lambda_2 t}$$

- This is an overdamped response.
- Mass decreases to equilibrium (0) very slowly.



Underdamped Response

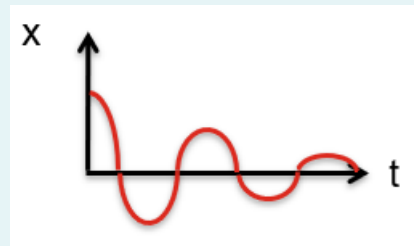
- If $b^2 < 4mk$, we will have complex roots.

$$\lambda_{1,2} = \frac{-b \pm \sqrt{b^2 - 4mk}}{2m} = p \pm qi$$

- The general solution of the 2nd order ODE is:

$$x(t) = c_1 e^{pt} \cos(qt) + c_2 e^{pt} \sin(qt)$$

- This is an underdamped response.
- Oscillatory response with reducing amplitude.



Critically Damped Response

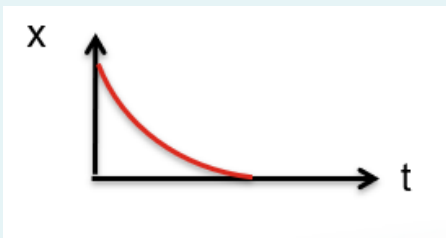
- If $b^2 = 4mk$, we will have real and equal roots.

$$\lambda_{1,2} = \lambda = \frac{-b}{2m}$$

- The general solution of the 2nd order ODE is:

$$x(t) = (c_1 + c_2 t)e^{\lambda t}$$

- This is an critically damped response.
- System reaches equilibrium position rapidly and without oscillation! Highly desirable!



Calculating c_1 and c_2 (1)

- In the previous three slides, the general solutions contain two parameters, c_1 and c_2 .
 - They can be found by using the **initial conditions** of the system.
- E.g. Find the response of the mass-spring-damper system for $m = 1, b = 5, k = 6$, when the block is **released from rest** and **from position** $x = -1$.
- Since $b^2 > 4mk$, we have real and unequal roots:

$$\lambda_1 = \frac{-b + \sqrt{b^2 - 4mk}}{2m} = -2, \lambda_2 = \frac{-b - \sqrt{b^2 - 4mk}}{2m} = -3$$

Calculating c_1 and c_2 (2)

- The general solution is therefore:

$$x(t) = c_1 e^{-2t} + c_2 e^{-3t}$$

- The velocity is:

$$\dot{x}(t) = -2c_1 e^{-2t} - 3c_2 e^{-3t}$$

- Using initial conditions, we have:

$$\begin{aligned} x(0) &= c_1 e^0 + c_2 e^0 = c_1 + c_2 = -1 \\ \dot{x}(0) &= -2c_1 e^0 - 3c_2 e^0 = -2c_1 - 3c_2 = 0 \end{aligned}$$

- This gives $c_1 = -3$ and $c_2 = 2$. The complete solution is:

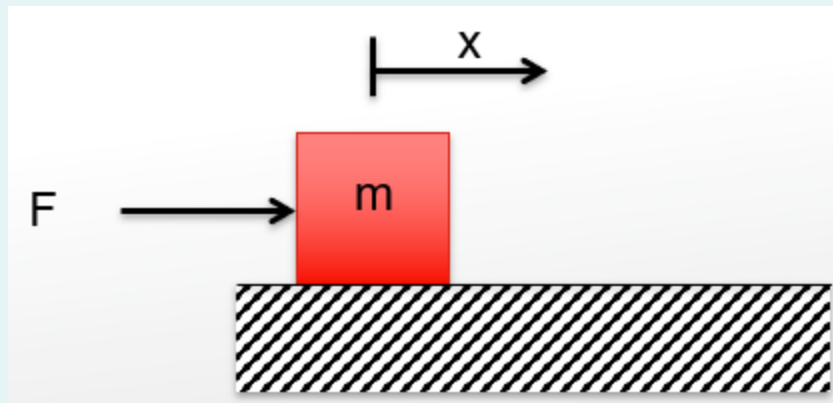
$$x(t) = -3e^{-2t} + 2e^{-3t}$$

Content

- Introduction
- Second Order Linear Systems
- **Control of Second Order Linear Systems**
- Control Law Partitioning
- Trajectory Following
- Modeling and Control of a Single Joint
- Modeling and Control of Complete Manipulator
- Matlab Simulation
- Tutorial Questions

Control of a Mass (1)

- Imagine you have a **mass** on a frictionless surface.
- You would like to bring it to a desired position by a **force**.



- How should the force be?

Control of a Mass (2)

- If we design the force such that it is proportional to x and $\dot{x}$, the system will behave like the mass-spring-damper system!
- Newtons' law for mass:

$$m\ddot{x} = F$$

- Force is designed as:

$$F = -k_v\dot{x} - k_p x$$

- The controlled system becomes:

$$m\ddot{x} = -k_v\dot{x} - k_p x \Rightarrow m\ddot{x} + k_v\dot{x} + k_p x = 0$$

Control of a Mass (3)

- The last equation has the same form of the natural system:

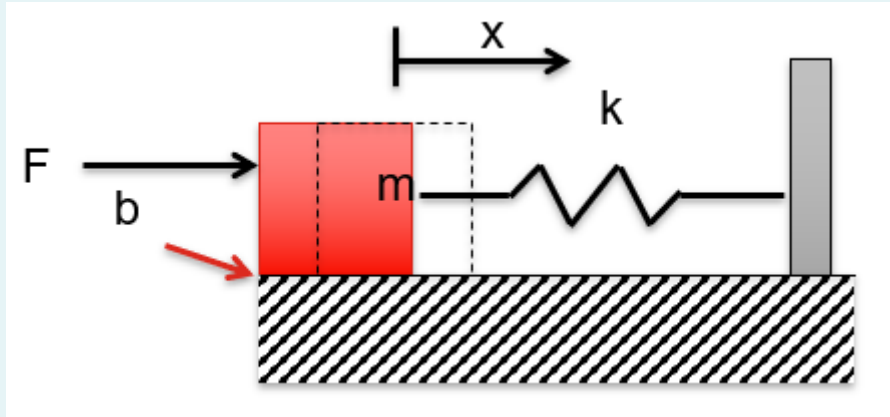
$$m\ddot{x} + k_v\dot{x} + k_px = 0 \leftrightarrow m\ddot{x} + b\dot{x} + bx = 0$$

- Earlier, we mentioned that the **critically damped response** ($b^2 = 4mk$) is the preferred response.
- The choice of k_v and k_p should therefore be:

1. Set any k_p according to desired stiffness of system.
2. Calculate k_v from $k_v^2 = 4mk_p$, or $k_v = 2\sqrt{mk_p}$.

Control of a Mass-Spring-Damper (1)

- Now, imagine you have a mass-spring-damper system:



- However, the relationship between m, b, k are not of critical damping.
 - E.g. b is too small so the response is oscillatory.

Control of a Mass-Spring-Damper (2)

- We can again use the force F to create a critically damped response.
- Newtons' law for mass:

$$m\ddot{x} = -b\dot{x} - kx + F$$

- Force is designed as:

$$F = -k_v\dot{x} - k_px$$

- The **controlled system** becomes:

$$m\ddot{x} = -b\dot{x} - kx - k_v\dot{x} - k_px \Rightarrow m\ddot{x} + (b + k_v)\dot{x} + (k + k_p)x = 0$$

Control of a Mass-Spring-Damper (3)

- The last equation is repeated here:

$$m\ddot{x} + (b + k_v)\dot{x} + (k + k_p)x = 0$$

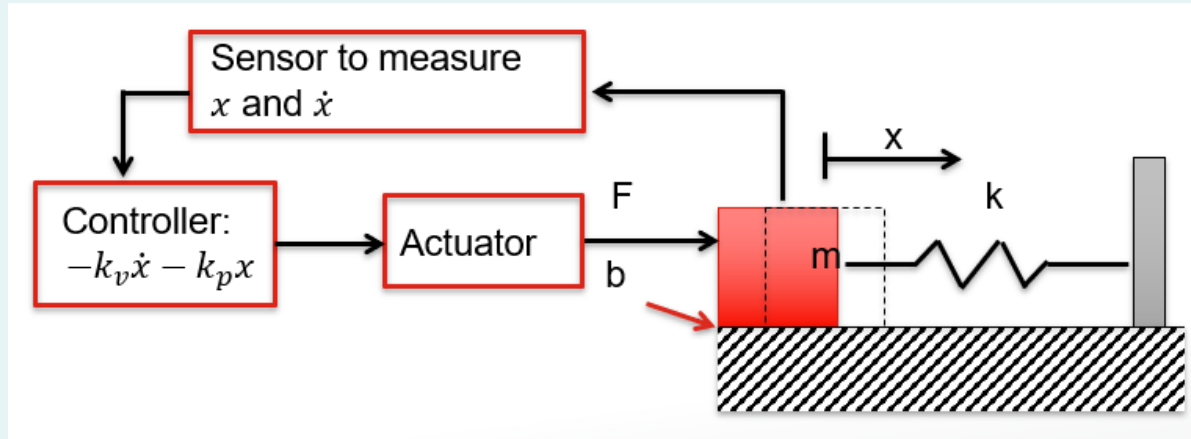
- To obtain critically damped response:

1. Set k_p such that $(k + k_p)$ matches desired stiffness of system.
2. Calculate k_v from $(b + k_v)^2 = 4m(k + k_p)$, or

$$k_v = 2\sqrt{m(k + k_p)} - b$$

Control of a Mass-Spring-Damper (4)

- The control implementation is as follows:



- Note: This controller is called the **Proportional-Derivative (PD) controller**.

Content

- Introduction
- Second Order Linear Systems
- Control of Second Order Linear Systems
- **Control Law Partitioning**
- Trajectory Following
- Modeling and Control of a Single Joint
- Modeling and Control of Complete Manipulator
- Matlab Simulation
- Tutorial Questions

Control Law Partitioning (1)

- We saw in the previous section that k_v and k_p need to be changed based on the system dynamics (m, b, k) .
- Is there a way to make k_v and k_p independent of the system parameters?
 - Yes! We partition the controller into two portions.
 - The first portion is a **model-based compensator**. This cancels off the system dynamics and reduces the system to a unit mass.
 - The second portion is the **servo controller**, where k_v and k_p will be independent of m, b, k .

Control Law Partitioning (2)

- The **procedure** is as follows. Given system dynamics:

$$m\ddot{x} + b\dot{x} + kx = F$$

- Design the **model based compensator** as:

$$F = m\underline{f} + b\dot{x} + kx$$

- The system is reduced to unit mass:

$$\ddot{x} = f$$

- The **servo portion** is then designed as:

$$f = -k_v\dot{x} - k_px$$

- Where k_p determines the stiffness, and $k_v = 2\sqrt{k_p}$.

Control Law Partitioning (3)

- How does the closed loop equation look like?

$$\begin{aligned} m\ddot{x} + b\dot{x} + kx &= F \\ &= m\bar{f} + b\dot{x} + kx \\ &= m(-k_v\dot{x} - k_px) + b\dot{x} + kx \end{aligned}$$

- The system thus reduces to:

$$\ddot{x} + k_v\dot{x} + k_px = 0$$

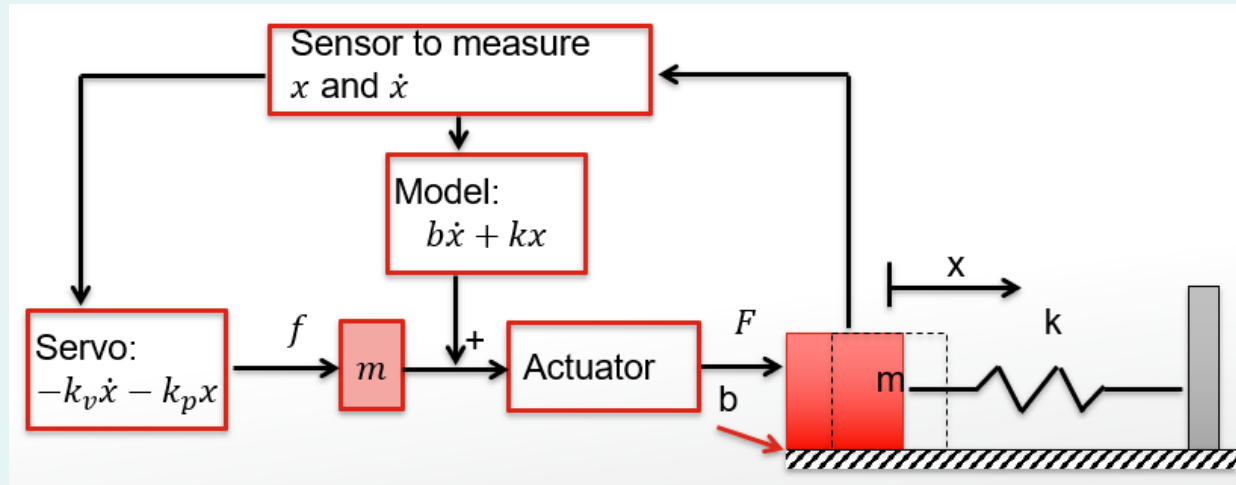
- which, with the choice of k_v and k_p , will provide critically damped response.

Control Law Partitioning (4)

- E.g. $m = 10, b = 20, k = 10$. Find gains k_v, k_p such the system becomes critically damped with stiffness of 16.
- System: $10\ddot{x} + 20\dot{x} + 10x = F$
- Design model based compensator: $F = 10\underline{f} + 20\dot{x} + 10x$
- Design servo controller: $f = -k_v\dot{x} - k_px$
- Let $k_p = 16$.
- Then $k_v = 2\sqrt{16} = 8$

Control Law Partitioning (5)

- The control implementation is as follows:



- We will use the control law partitioning technique for **all subsequent control design in this lecture.**

Control Law Partitioning (6)

- You might be wondering: How does the model-based compensator cancels off system dynamics? How is this even possible?
- Actually.. If you look at the control law carefully:

$$\begin{aligned} F &= m(-k_v\dot{x} - k_px) + b\dot{x} + kx \\ &= (-mk_v + b)\dot{x} + (-mk_p + k)x \end{aligned}$$

- The controller is still a **PD-type controller**!
 - It's just that the control-law partitioning technique makes the **design so much easier**!

Content

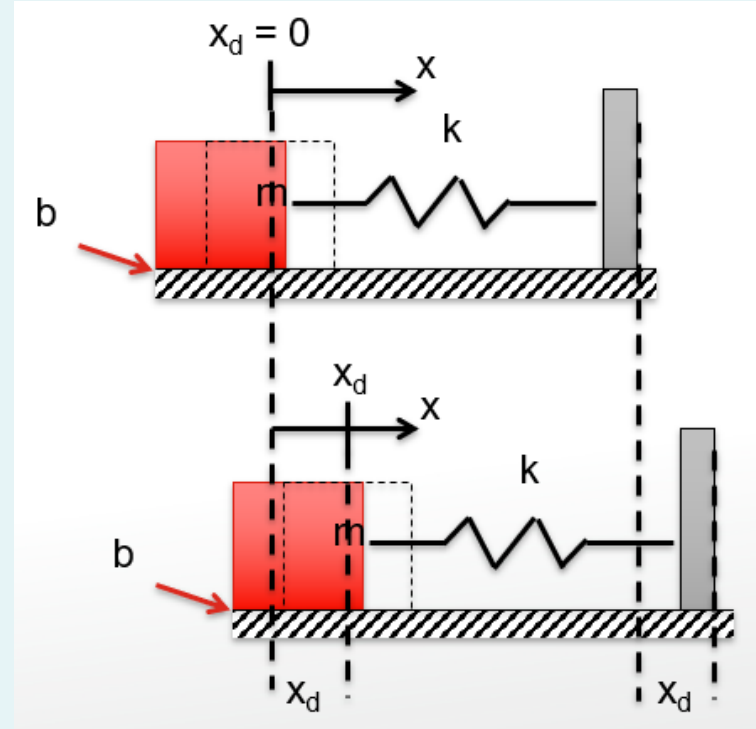
- Introduction
- Second Order Linear Systems
- Control of Second Order Linear Systems
- Control Law Partitioning
- **Trajectory Following**
- Modeling and Control of a Single Joint
- Modeling and Control of Complete Manipulator
- Matlab Simulation
- Tutorial Questions

Regulation to Another Position (1)

- So far, we have only looked at maintaining the block of mass at the **zero position** ($x_d = 0$).
- What if we want the mass to move to **another position**, $x_d \neq 0$?

Regulation to Another Position (2)

- Let's think about this for a natural second order system:
- If we **offset** the grey “wall” by a distance x_d ,
- the **equilibrium position** of the mass-spring-damper also moves by a distance x_d .
- Thus the mass will move to the desired position x_d .



Regulation to Another Position (3)

- Mathematically, this means $m\ddot{x} = -b\dot{x} - k(\underline{x - x_d})$.
- To achieve regulation using control follows similar idea.

- System: $m\ddot{x} + b\dot{x} + kx = F$
- Design model based compensator: $F = m\underline{f} + b\dot{x} + kx$
- Design servo controller: $f = -k_v\dot{x} - k_p(\underline{x - x_d})$
- Set the desired stiffness k_p .
- Then $k_v = 2\sqrt{k_p}$

Trajectory Following (1)

- The same idea can also be expanded to **trajectory following**, i.e. the **target position keeps moving** and is not stationary.

- System: $m\ddot{x} + b\dot{x} + kx = F$
- Design model based compensator: $F = m\underline{\ddot{f}} + b\underline{\dot{x}} + k\underline{x}$
- Design servo controller: $\underline{f} = \underline{\ddot{x}_d} - k_v(\underline{\dot{x}} - \underline{\dot{x}_d}) - k_p(\underline{x} - \underline{x_d})$
- Set the desired stiffness k_p .
- Then $k_v = 2\sqrt{k_p}$

Trajectory Following (2)

- Why would this work?
- The closed loop equation using the designed controller is:

$$\begin{aligned} m\ddot{x} + b\dot{x} + kx &= F \\ &= m\underline{f} + b\dot{x} + kx \\ &= m\left(\underline{\ddot{x}}_d - k_v(\underline{\dot{x}} - \underline{\dot{x}}_d) - k_p(\underline{x} - \underline{x}_d)\right) + b\dot{x} + kx \end{aligned}$$

- Or:

$$(\ddot{x}_d - \ddot{x}) + k_v(\dot{x}_d - \dot{x}) + k_p(x_d - x) = 0$$

Trajectory Following (3)

- If we define **tracking error** as $e = x_d - x$
- The closed loop equation then becomes:
$$\ddot{e} + k_v \dot{e} + k_p e = 0$$
- e has the **same differential equation** as discussed earlier!
- Since e decreases to zero, **x will track x_d** eventually.

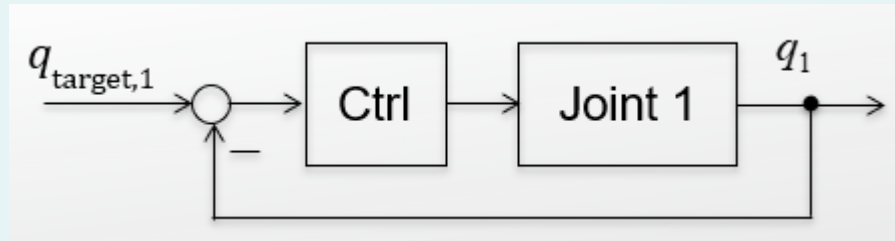


Content

- Introduction
- Second Order Linear Systems
- Control of Second Order Linear Systems
- Control Law Partitioning
- Trajectory Following
- **Modeling and Control of a Single Joint**
- Modeling and Control of Complete Manipulator
- Matlab Simulation
- Tutorial Questions

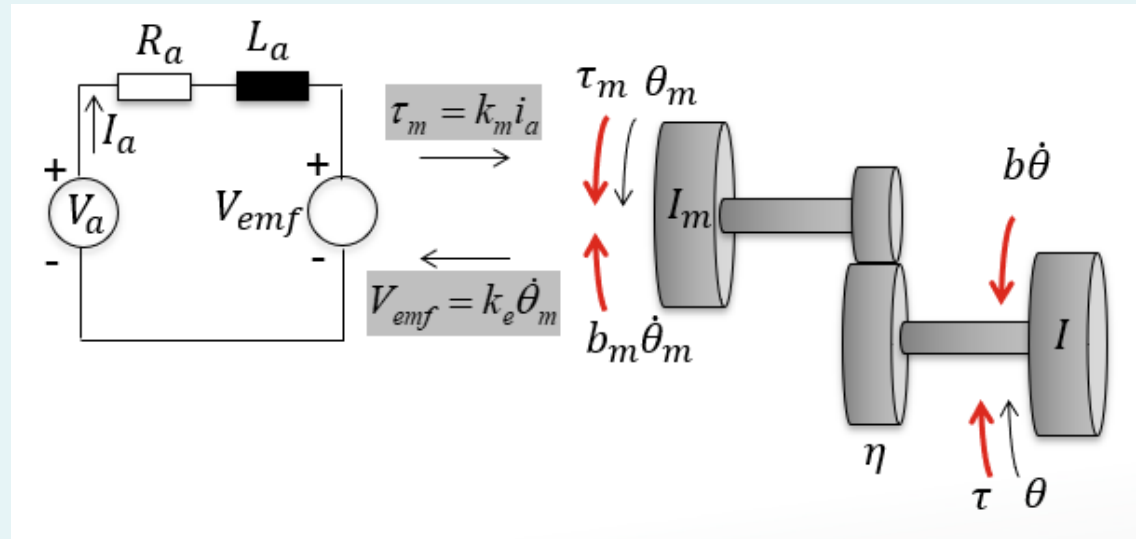
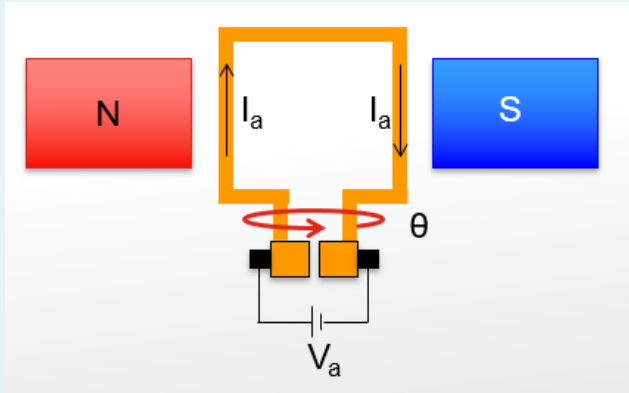
Modeling of a Single Joint (1)

- The control law partitioning technique is very powerful.
- If we have the system's model, we can design the controller very easily.
- We will start with obtaining the **model of each joint** for the **individual joint control**.



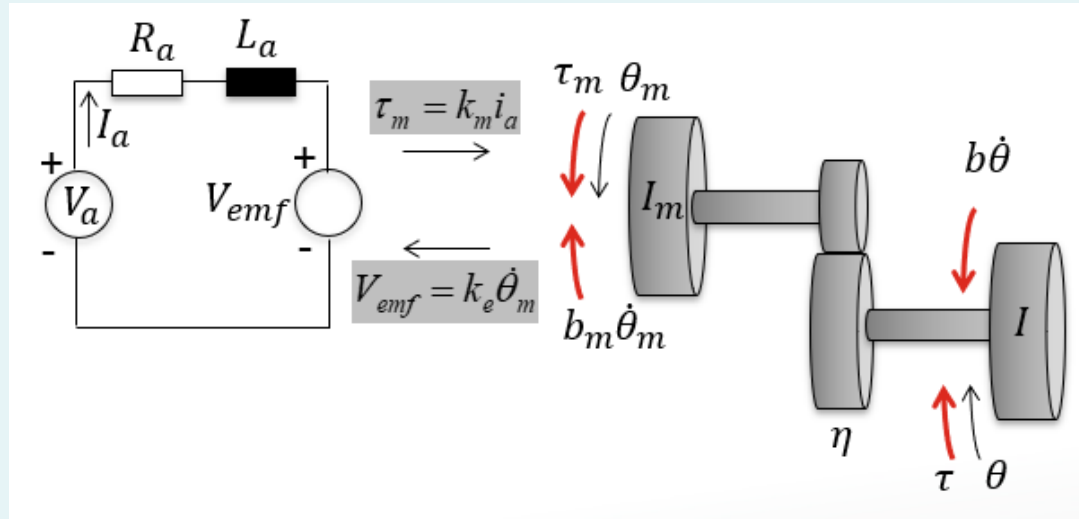
Modeling of a Single Joint (2)

- Each joint consists of two parts: **Electrical** (e.g. DC motor) and **mechanical** (Shaft, gear, load).



Modeling of a Single Joint (3)

- The **electrical** part creates a torque for the mechanical part, but the **mechanical** part also induces a back EMF to the electrical part.



Modeling of a Single Joint (4)

- Due to the interaction between stator field and armature current, the torque generated by a PM DC motor is proportional to the armature current:

$$\tau_m = k_m i_a$$

- Where k_m is the torque constant of the motor.
- The back EMF is proportional to rotor speed:

$$V_{emf} = k_e \dot{\theta}_m$$

- Where k_e is the electrical constant of the motor.

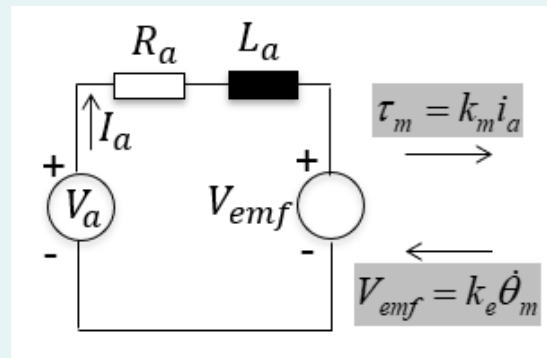
Modeling of a Single Joint (5)

- The electric circuit equation is:

$$\begin{aligned} L_a \frac{di_a}{dt} + R_a i_a &= V_a - V_{emf} \\ &= V_a - k_e \dot{\theta}_m \end{aligned}$$

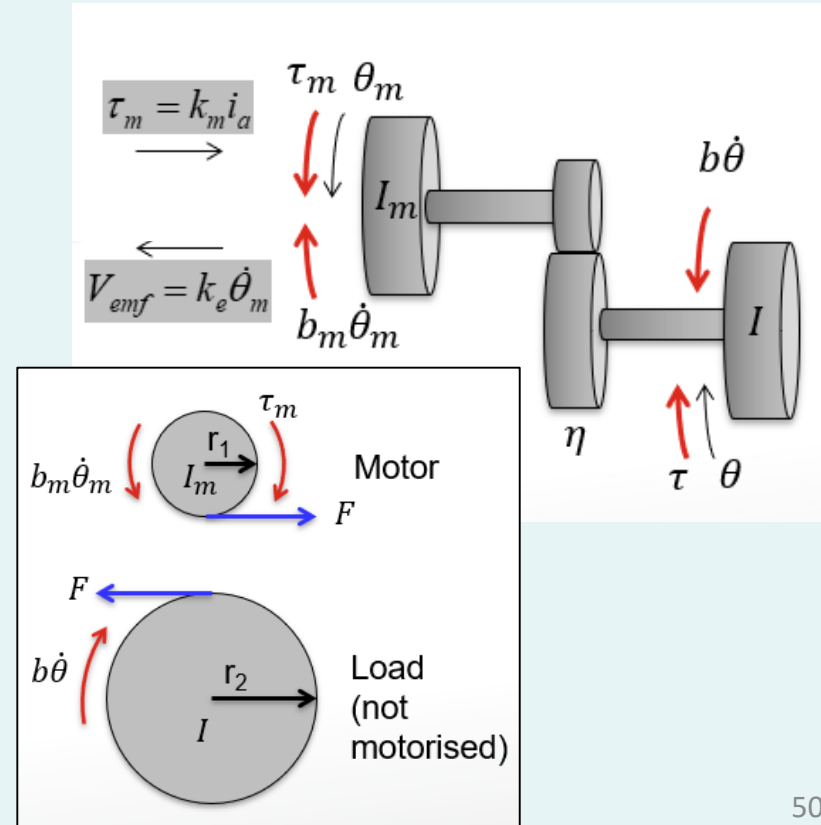
- Assumption: The Inductance L_a is negligible, thus:

$$R_a i_a = V_a - k_e \dot{\theta}_m$$



Modeling of a Single Joint (6)

- The **mechanical equation** can be derived from the free body diagram below.
- Note, the motor-side gear **pushes** the load-side gear with a force F .
- The load-side gear also creates an equal but opposite **reaction force** onto motor-side gear.



Modeling of a Single Joint (7)

- Motor side dynamics:

$$I_m \ddot{\theta}_m = \tau_m - b_m \dot{\theta}_m - Fr_1 \quad (1)$$

- Load side dynamics:

$$I \ddot{\theta} = Fr_2 - b \dot{\theta} \quad (2)$$

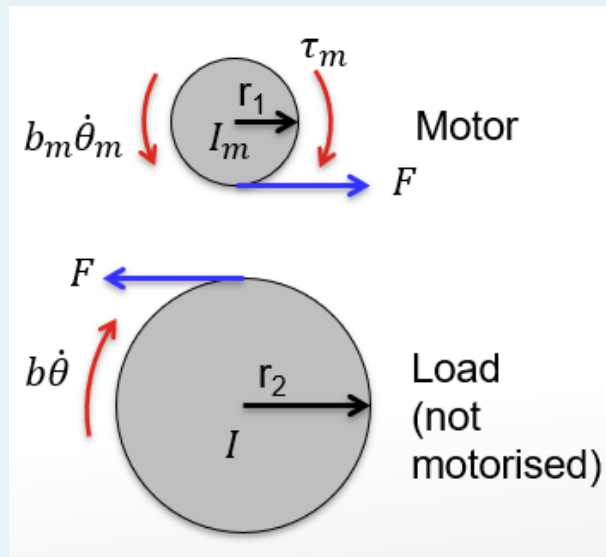
- If we call:

$$\tau = Fr_2 \quad (3)$$

- Then (2) becomes:

$$I \ddot{\theta} = Fr_2 - b \dot{\theta} = \tau - b \dot{\theta}$$

$$\tau = I \ddot{\theta} + b \dot{\theta} \quad (4)$$



Modeling of a Single Joint (8)

- Equation (3) also gives:

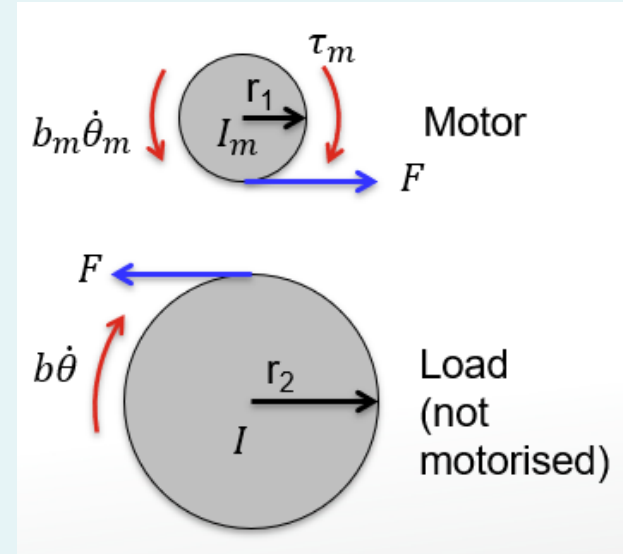
$$F = \frac{\tau}{r_2}$$

- Equation (1) can now be written as:

$$\begin{aligned} I_m \ddot{\theta}_m &= \tau_m - b_m \dot{\theta}_m - F r_1 \\ &= \tau_m - b_m \dot{\theta}_m - \frac{r_1}{r_2} \tau \end{aligned} \quad (5)$$

- Substituting (4) into (5) gives:

$$I_m \ddot{\theta}_m = \tau_m - b_m \dot{\theta}_m - \frac{r_1}{r_2} (I \ddot{\theta} + b \dot{\theta}) \quad (6)$$



Modeling of a Single Joint (9)

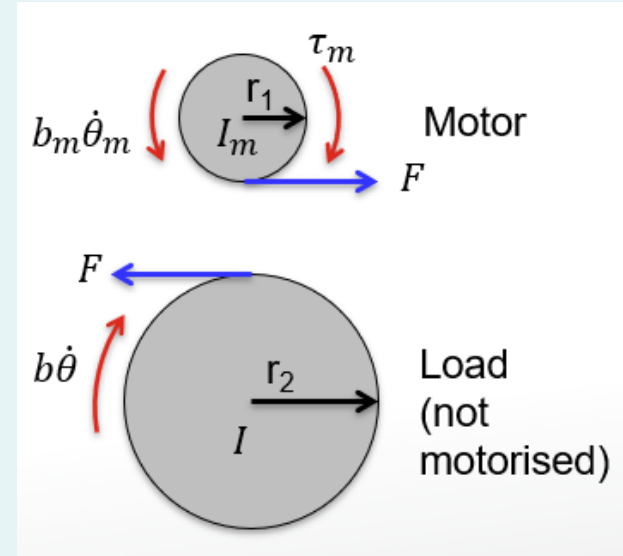
- We want to express equation (6) solely using **load-side terms**.
- Define **gear ratio**:

$$\eta = \frac{r_2}{r_1} \quad (7)$$

- The gear ratio increases the torque seen at the load, but reduces the speed of the load:

$$\tau = \eta \tau_m \quad (8)$$

$$\dot{\theta} = \frac{1}{\eta} \dot{\theta}_m \quad (9)$$



Modeling of a Single Joint (10)

- Substituting (7), (8), (9) into (6) gives:

$$I_m \eta \ddot{\theta} = \frac{1}{\eta} \tau - b_m \eta \dot{\theta} - \frac{1}{\eta} (I \ddot{\theta} + b \dot{\theta})$$

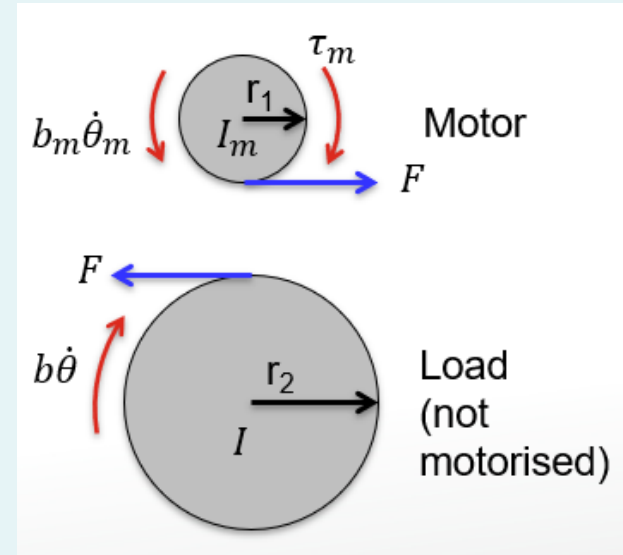
- Or:

$$(I + I_m \eta^2) \ddot{\theta} + (b + b_m \eta^2) \dot{\theta} = \tau \quad (10)$$

- Equation (10) has the form of

$$m \ddot{x} + b \dot{x} + kx = F$$

- And will be used for control design.



Control of a Single Joint (1)

- As a summary, the dynamics of the load is:

$$(I + I_m \eta^2) \ddot{\theta} + (b + b_m \eta^2) \dot{\theta} = \tau$$

- We can now use the **control law partitioning** technique to control the single joint:

- Design model-based control: $\tau = (I + I_m \eta^2) \underline{f} + (b + b_m \eta^2) \dot{\theta}$
- Design servo controller: $\underline{f} = \ddot{\theta}_d - k_v (\dot{\theta} - \dot{\theta}_d) - k_p (\theta - \theta_d)$
- Set the desired stiffness k_p .
- Then $k_v = 2\sqrt{k_p}$

Control of a Single Joint (2)

- The control law computes the required torque on the load side, τ .
- However, it needs to be provided by the motor.
- We know $\tau = \eta \tau_m$, so the required motor side torque is:

$$\tau_m = \frac{1}{\eta} \tau$$

- To generate this motor torque, we control the current:

$$\tau_m = k_m i_a$$

- Which is achieved by varying the voltage $R_a i_a = V_a - k_e \dot{\theta}_m$.

Video Example (1)

- Single joint control – Overdamped response.



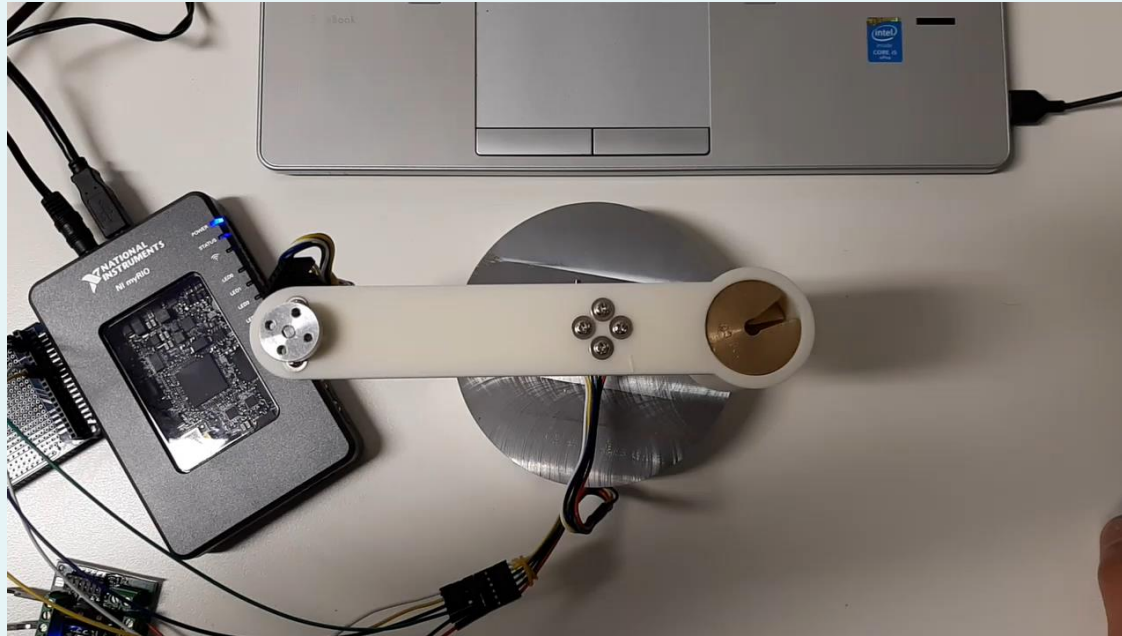
Video Example (2)

- Single joint control – Underdamped response.



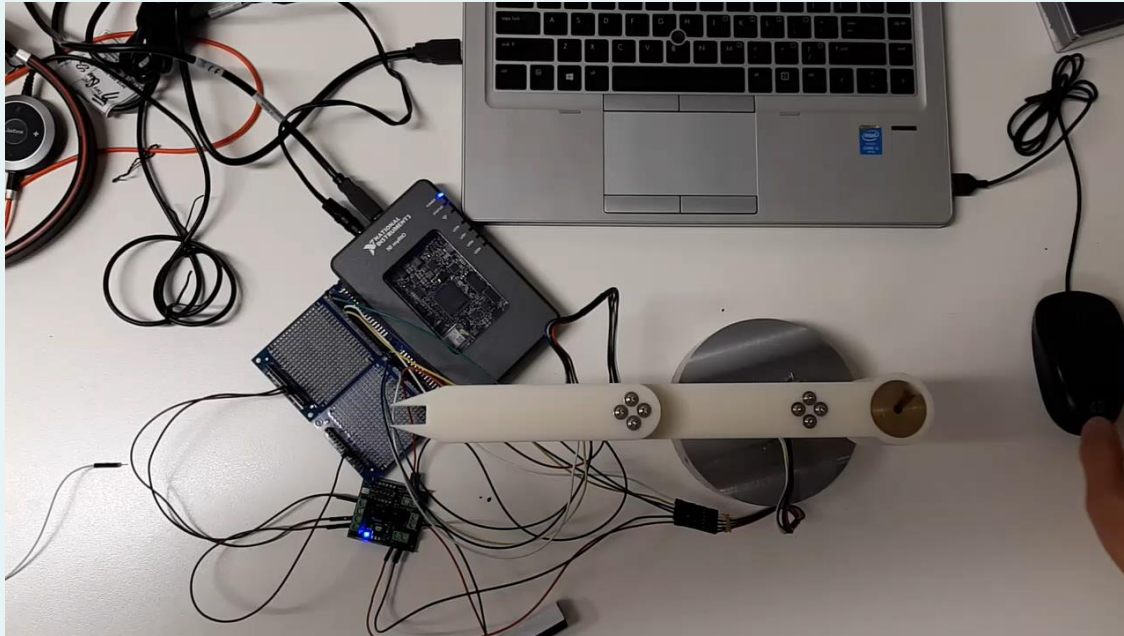
Video Example (3)

- Single joint control – Critically damped response.



Video Example (4)

- Single joint control but on both joints.

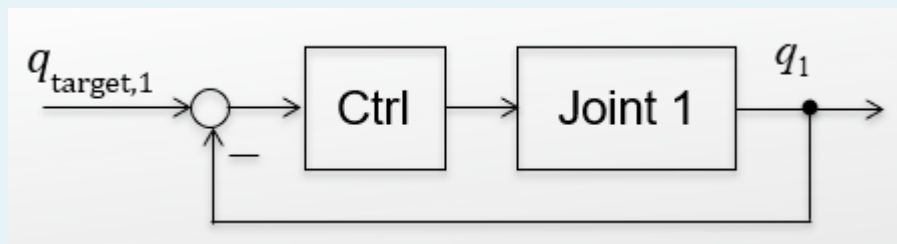


Content

- Introduction
- Second Order Linear Systems
- Control of Second Order Linear Systems
- Control Law Partitioning
- Trajectory Following
- Modeling and Control of a Single Joint
- **Modeling and Control of Complete Manipulator**
- Matlab Simulation
- Tutorial Questions

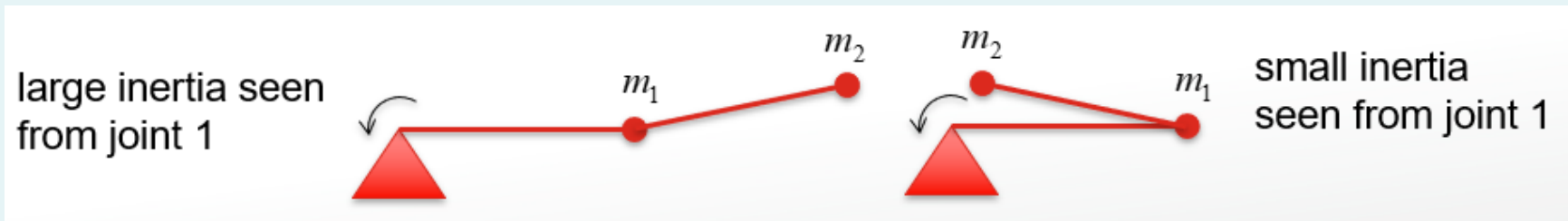
Control of a Complete Manipulator (1)

- In previous section, we assumed that:
 - Each joint is independent of other joints.
 - The inertia “seen” by each joint actuator is constant.
- These assumptions allowed us to make **linear analysis** and design **linear controller** for each joint.



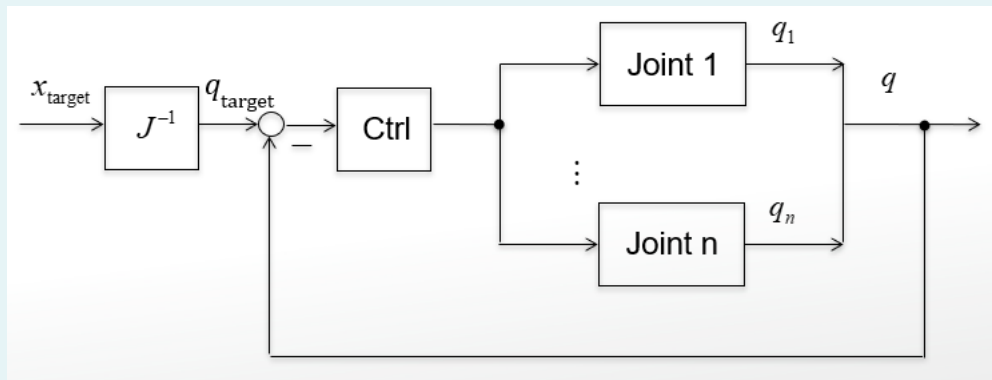
Control of a Complete Manipulator (2)

- Unfortunately, the assumptions are not true for a real manipulator.
 - The joints or links are highly coupled.
 - The inertia (and other) matrices are NOT constant.



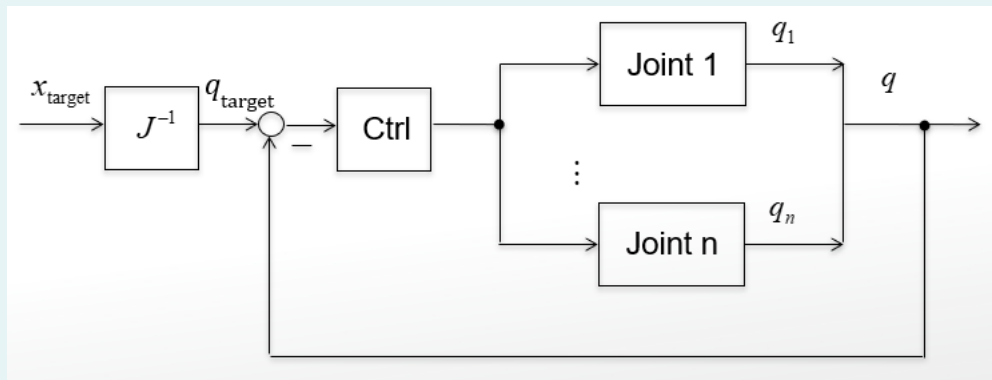
Control of a Complete Manipulator (3)

- The use of linear control will therefore lead to **undesirable results**.
 - E.g. the damping will NOT be uniform throughout the workspace.
- We will now design the controller for the robot as a **multi-input-multi-output** system, instead of individual joints.



Control of a Complete Manipulator (4)

- Good news! This is very easy using the **control law partitioning** technique!
- The model-based compensator will cancel off the robot's dynamics and decouple each joint.
- Servo controllers will then be designed for each joint individually.



Control of a Complete Manipulator (5)

- The dynamic model of a manipulator is:

$$M(q)\ddot{q} + V(q, \dot{q}) + G(q) = \tau$$

- Design the **model-based compensator** as:

$$\tau = M(q)\underline{f} + V(q, \dot{q}) + G(q)$$

- This will decouple all the individual joint dynamics:

$$\ddot{q} = f$$

- Or:

$$\begin{bmatrix} \ddot{q}_1 \\ \ddot{q}_2 \\ \vdots \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ \vdots \end{bmatrix}$$

Control of a Complete Manipulator (6)

- The **servo controller** f is then designed as:

$$f = \ddot{q}_d - K_v(\dot{q} - \dot{q}_d) - K_p(q - q_d)$$

- Where K_v and K_p are matrices.
- Or more explicitly:

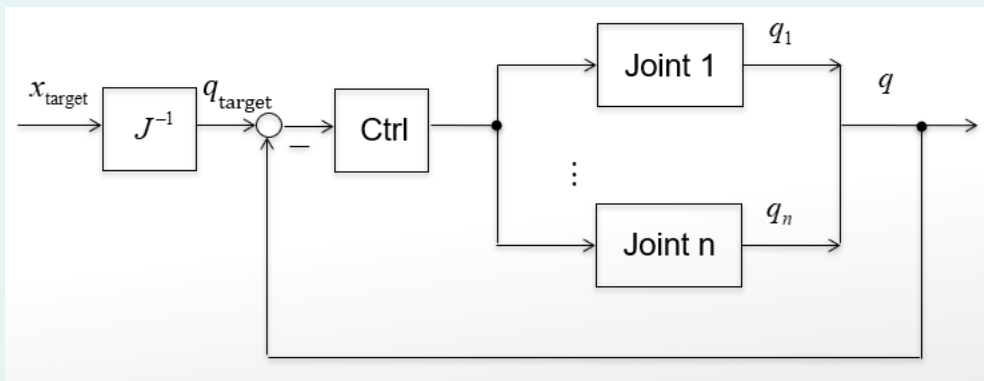
$$\begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ \vdots \end{bmatrix} = \begin{bmatrix} \ddot{q}_{d1} \\ \ddot{q}_{d2} \\ \ddot{q}_{d3} \\ \vdots \end{bmatrix} - \begin{bmatrix} k_{v1} & 0 & 0 & \cdots \\ 0 & k_{v2} & 0 & \cdots \\ 0 & 0 & k_{v3} & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} \begin{bmatrix} \dot{q}_1 - \dot{q}_{d1} \\ \dot{q}_2 - \dot{q}_{d2} \\ \dot{q}_3 - \dot{q}_{d3} \\ \vdots \end{bmatrix} - \begin{bmatrix} k_{p1} & 0 & 0 & \cdots \\ 0 & k_{p2} & 0 & \cdots \\ 0 & 0 & k_{p3} & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} \begin{bmatrix} q_1 - q_{d1} \\ q_2 - q_{d2} \\ q_3 - q_{d3} \\ \vdots \end{bmatrix}$$

Content

- Introduction
- Second Order Linear Systems
- Control of Second Order Linear Systems
- Control Law Partitioning
- Trajectory Following
- Modeling and Control of a Single Joint
- Modeling and Control of Complete Manipulator
- **Matlab Simulation**
- Tutorial Questions

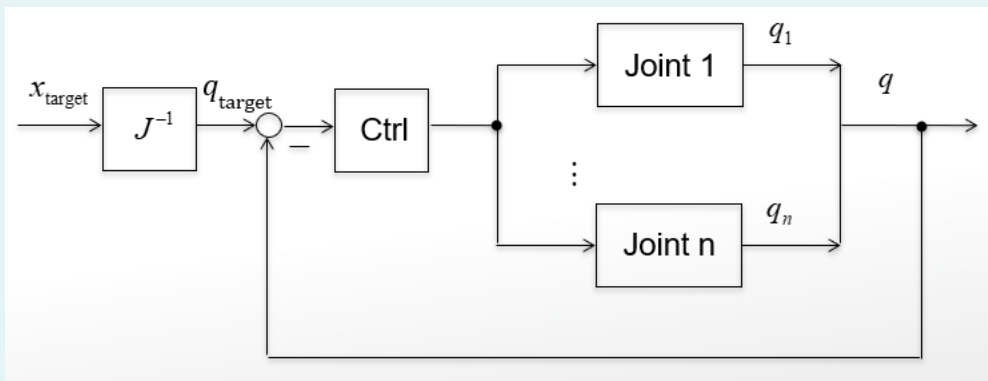
Matlab Simulation (1)

- In the past several weeks, we have written codes to:
 - Simulate robot motion given pre-set torques.
 - Generate trajectory.
- We will now combine them, but the pre-set torques shall be replaced by torques computed by the controller.



Matlab Simulation (2)

- We will also focus on the “complete manipulator” approach since we have the full robot model.
- In the given code, the trajectory is planned in Cartesian space, thus inverse kinematics is required.
- You may also plan the trajectory directly in joint space.



Matlab Simulation (3)

- Set robot and control parameters:

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Robot Parameters %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

m1 = 3;    % kg
m2 = 2;    % m
L1 = 3;    % kg
L2 = 2;    % m
g = 9.8;   % m/s^2

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Control Parameters %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Kp = diag([100 100]); % diagonal matrix Kp
Kv = diag([2*sqrt(100) 2*sqrt(100)]); % diagonal matrix Kv
                                     % Critical damping. You may try other values to see
                                     % different responses.
```

Matlab Simulation (4)

- Initial and target positions for x and y.

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Boundary Conditions          %
% Must be within Workspace %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

x0 = 2; % x initial
y0 = 3; % y initial
xf = 4; % x target
yf = 1; % y target
tf = 3;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Create Time Array %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

T = 1e-3; % time interval for integration
t = 0:T:tf; % array of time from 0 to tf in steps of 0.001
t = t'; % make the time array into column vector
```


Matlab Simulation (5)

- Generate Quintic trajectory for x.

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Quintic Polynomial for x %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

a0 = x0;
a1 = 0;
a2 = 0;
a3 = 10/tf^3*(xf-x0);
a4 = -15/tf^4*(xf-x0);
a5 = 6/tf^5*(xf-x0);

xQuintic = a0*ones(length(t),1)+a1*t+a2*t.^2+a3*t.^3+a4*t.^4+a5*t.^5;
xDotQuintic = a1*ones(length(t),1)+2*a2*t+3*a3*t.^2+4*a4*t.^3+5*a5*t.^4;
xDoubleDotQuintic = 2*a2*ones(length(t),1)+6*a3*t+12*a4*t.^2+20*a5*t.^3;

figure,sgtitle('x Quintic')
subplot(1,3,1),plot(t,xQuintic),title('Position')
subplot(1,3,2),plot(t,xDotQuintic),title('Velocity')
subplot(1,3,3),plot(t,xDoubleDotQuintic),title('Acceleration')
```

Matlab Simulation (6)

- Generate Quintic trajectory for y.

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Quintic Polynomial for y %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

a0 = y0;
a1 = 0;
a2 = 0;
a3 = 10/tf^3*(yf-y0);
a4 = -15/tf^4*(yf-y0);
a5 = 6/tf^5*(yf-y0);

yQuintic = a0*ones(length(t),1)+a1*t+a2*t.^2+a3*t.^3+a4*t.^4+a5*t.^5;
yDotQuintic = a1*ones(length(t),1)+2*a2*t+3*a3*t.^2+4*a4*t.^3+5*a5*t.^4;
yDoubleDotQuintic = 2*a2*ones(length(t),1)+6*a3*t+12*a4*t.^2+20*a5*t.^3;

figure,sgtitle('y Quintic')
subplot(1,3,1),plot(t,yQuintic),title('Position')
subplot(1,3,2),plot(t,yDotQuintic),title('Velocity')
subplot(1,3,3),plot(t,yDoubleDotQuintic),title('Acceleration')
```

Matlab Simulation (7)

- Use inverse kinematics to get joint trajectories.

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Inverse Kinematics at Each Step %
% See notes of Geometrical Method %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

for i = 1:length(t)
    theta2Quintic(i) = acos((xQuintic(i)^2+yQuintic(i)^2-L1^2-L2^2)/(2*L1*L2));
    beta = atan2(yQuintic(i),xQuintic(i));
    psi = acos((L1^2+xQuintic(i)^2+yQuintic(i)^2-L2^2)/(2*L1*sqrt(xQuintic(i)^2+yQuintic(i)^2)));
    theta1Quintic(i) = beta - psi;
end

figure,sgtitle('Theta Quintic')
subplot(1,2,1),plot(t,theta1Quintic),title('Theta 1')
subplot(1,2,2),plot(t,theta2Quintic),title('Theta 2')
```

Matlab Simulation (8)

- Robot initial conditions and placeholder.

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Robot Initial Conditions %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

q1 = theta1Quintic(1); % joint 1 angle
q1Dot = 0; % joint 1 angular velocity
q2 = theta2Quintic(1); % joint 2 angle
q2Dot = 0; % joint 2 angular velocity

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Variable Declaration and Memory Placeholder %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

q1DoubleDotRecord = [];
q2DoubleDotRecord = [];
q1DotRecord = [];
q2DotRecord = [];
q1Record = [];
q2Record = [];
q1dot = 0;
q2dot = 0;
```

Matlab Simulation (9)

- Calculate robot dynamics.

```
for i=1:length(t)

    %%%%%%%%%%%%%%%
    % Calculate M, V, G %
    %%%%%%%%%%%%%%%

    m11 = m2*L2^2 + 2*m2*L1*L2*cos(q2) + (m1+m2)*L1^2;
    m12 = m2*L2^2 + m2*L1*L2*cos(q2);
    m21 = m12;
    m22 = m2*L2^2;
    M = [m11,m12;m21,m22];

    v1 = -m2*L1*L2*sin(q2)*q2Dot^2 - 2*m2*L1*L2*sin(q2)*q1Dot*q2Dot;
    v2 = m2*L1*L2*sin(q2)*q1Dot^2;
    V = [v1;v2];

    g1 = m2*g*L2*cos(q1+q2) + (m1+m2)*g*L1*cos(q1);
    g2 = m2*g*L2*cos(q1+q2);
    G = [g1;g2];
```

Matlab Simulation (10)

- Calculate control.

```
%%%%%%%%%%%%%%  
% Calculate Control %  
%%%%%%%%%%%%%%  
  
servoControl = -Kv*[q1dot;q2dot] - Kp*[q1 - theta1Quintic(i); q2 - theta2Quintic(i)];  
% Note velocity and acceleration trajectory are set as zero  
% Because we are doing regulation, not trajectory tracking  
  
modelControl = M*servoControl + V + G;  
  
Tau1(i) = modelControl(1);  
Tau2(i) = modelControl(2);
```

Matlab Simulation (11)

- Calculate acceleration and velocity due to the control.

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Calculate Acceleration %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

qDoubleDot = inv(M) * ([Tau1(i);Tau2(i)]-V-G);
q1DoubleDot = qDoubleDot(1);
q2DoubleDot = qDoubleDot(2);

q1DoubleDotRecord = [q1DoubleDotRecord;q1DoubleDot];
q2DoubleDotRecord = [q2DoubleDotRecord;q2DoubleDot];

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Calculate Velocity %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

q1Dot = q1Dot + q1DoubleDot*T;
q2Dot = q2Dot + q2DoubleDot*T;

q1DotRecord = [q1DotRecord;q1Dot];
q2DotRecord = [q2DotRecord;q2Dot];
```

Matlab Simulation (12)

- Update positions.

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Calculate Position %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

q1 = q1 + q1Dot*T + 1/2*q1DoubleDot*T^2;
q2 = q2 + q2Dot*T + 1/2*q2DoubleDot*T^2;

q1Record = [q1Record;q1];
q2Record = [q2Record;q2];

end
```


Matlab Simulation (13)

- Simulate robot movement.

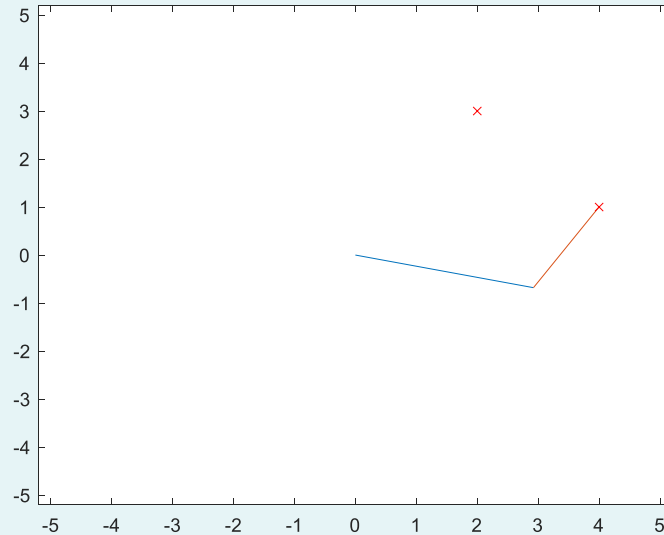
```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Show Robot Motion %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

x1 = L1*cos(q1Record);           % These are kinematic
y1 = L1*sin(q1Record);           % equation of the
x2 = x1 + L2*cos(q1Record+q2Record); % 2 link robot
y2 = y1 + L2*sin(q1Record+q2Record);

figure
for i = 1:100:length(x1) % Plot every 100th data
    clf; % Clear figure before new plot
    plot([0 x1(i)], [0 y1(i)]);
    axis([- (L1+L2+0.2) (L1+L2+0.2) - (L1+L2+0.2) (L1+L2+0.2)]);
    hold on, plot([x1(i) x2(i)], [y1(i) y2(i)]);
    hold on, plot(x0, y0, 'xr');
    hold on, plot(xf, yf, 'xr');
    pause(0.1);
end
```

Matlab Simulation (14)

- The robot simulation will be shown.



- Similar to last week, try coding movement of several segments.

Content

- Introduction
- Second Order Linear Systems
- Control of Second Order Linear Systems
- Control Law Partitioning
- Trajectory Following
- Modeling and Control of a Single Joint
- Modeling and Control of Complete Manipulator
- Matlab Simulation
- Tutorial Questions

Question 1

- Determine the motion of a mass-spring-damper system if parameter values are $m = 2$, $b = 6$ and $k = 4$, and the mass (initially at rest) is released from the position $x = 1$.

Question 2

- Determine the motion of a mass-spring-damper system if parameter values are $m = 1$, $b = 2$ and $k = 1$, and the mass (initially at rest) is released from the position $x = 4$.

Question 3

- Determine the motion of a mass-spring-damper system if parameter values are $m = 1$, $b = 4$ and $k = 5$, and the mass (initially at rest) is released from the position $x = 2$.

Question 4

- Give the nonlinear control equations for the system:

$$(2\sqrt{\theta} + 1)\ddot{\theta} + 3\dot{\theta}^2 - \sin(\theta) = \tau$$

- Choose gains so that this system is always critically damped with closed-loop stiffness of 10.

Question 5

- Design a trajectory-following control system for a system with the following dynamic equations:

$$\begin{aligned} m_1 l_1^2 \ddot{\theta}_1 + m_1 l_1 l_2 \dot{\theta}_1 \dot{\theta}_2 &= \tau_1 \\ m_2 l_2^2 (\ddot{\theta}_1 + \ddot{\theta}_2) + v_2 \dot{\theta}_2 &= \tau_2 \end{aligned}$$

- Additional question: What is wrong with robot model?

Thank you for your attention!

Any questions?